

PRACTICAL WEEK9

TASK1: Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
GNU nano 8.7.1 task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/stat.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    int src, dest;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read, bytes_written;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    src = open(argv[1], O_RDONLY);
    if (src < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    dest = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest < 0)
    {
        perror("Error opening destination file");
        close(src);
        return 1;
    }

    while ((bytes_read = read(src, buffer, BUFFER_SIZE)) > 0)
    {
        bytes_written = write(dest, buffer, bytes_read);
        if (bytes_written != bytes_read)
        {
            perror("Error writing file");
        }
    }
}
```

```

        close(src);
        close(dest);
        return 1;
    }
}

if (bytes_read < 0)
{
    perror("Error reading file");
}

lseek(src, 0, SEEK_SET);

close(src);
close(dest);

printf("File copied successfully using low-level I/O.\n");
return 0;
}

```

rishill10@ASUS16X: ~/Practica × + ▾

GNU nano 8.7.1 task2.c *

```

#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 4096

int main(int argc, char *argv[])
{
    FILE *src, *dest;
    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    src = fopen(argv[1], "rb");
    if (src == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    dest = fopen(argv[2], "wb");
    if (dest == NULL)
    {
        perror("Error opening destination file");
        fclose(src);
        return 1;
    }

    while ((bytes_read = fread(buffer, 1, BUFFER_SIZE, src)) > 0)
    {
        if (fwrite(buffer, 1, bytes_read, dest) != bytes_read)
        {
            perror("Error writing file");
            fclose(src);
            fclose(dest);
            return 1;
        }
    }
}

```

```
fclose(src);  
fclose(dest);  
  
printf("File copied successfully using standard library I/O.\n");  
return 0;  
}
```

```
rishill10@ASUS16X: ~/Practica X + v - □ X  
  
rishill10@ASUS16X:~$ mkdir Practical9  
cd Practical9  
rishill10@ASUS16X:~/Practical9$ nano task1.c  
rishill10@ASUS16X:~/Practical9$ nano task2.c  
rishill10@ASUS16X:~/Practical9$ echo "This is a sample file for testing file  
copy operations in Linux." > input.txt  
cat input.txt  
This is a sample file for testing file copy operations in Linux.  
rishill10@ASUS16X:~/Practical9$ gcc task1.c -o task1  
./task1 input.txt output_low.txt  
cat output_low.txt  
File copied successfully using low-level I/O.  
This is a sample file for testing file copy operations in Linux.  
rishill10@ASUS16X:~/Practical9$ gcc task2.c -o task2  
./task2 input.txt output_lib.txt  
cat output_lib.txt  
File copied successfully using standard library I/O.  
This is a sample file for testing file copy operations in Linux.  
rishill10@ASUS16X:~/Practical9$ cmp input.txt output_low.txt  
cmp input.txt output_lib.txt  
rishill10@ASUS16X:~/Practical9$ |
```

TASK2: Develop a program that redirects standard input and output using file descriptors and dup2(). Demonstrate how shells implement I/O redirection.

```
GNU nano 8.7.1                                redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int input_fd, output_fd;

    input_fd = open("input.txt", O_RDONLY);
    if (input_fd < 0)
    {
        perror("Error opening input file");
        return 1;
    }

    if (dup2(input_fd, STDIN_FILENO) < 0)
    {
        perror("dup2 input");
        return 1;
    }
    close(input_fd);

    output_fd = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (output_fd < 0)
    {
        perror("Error opening output file");
        return 1;
    }

    if (dup2(output_fd, STDOUT_FILENO) < 0)
    {
        perror("dup2 output");
        return 1;
    }
    close(output_fd);

    printf("This line is redirected to output.txt.\n");
    return 0;
}
```

```
rishill10@ASUS16X: ~/Practica × + v
GNU nano 8.7.1 read.c
#include <stdio.h>

int main()
{
    char str[100];

    scanf("%99s", str);

    printf("You entered: %s\n", str);

    return 0;
}
```

```
rishill10@ASUS16X: ~/Practica × + v - □ ×
rishill10@ASUS16X:~/Practical9$ nano redirect.c
rishill10@ASUS16X:~/Practical9$ nano read.c
rishill10@ASUS16X:~/Practical9$ gcc redirect.c -o redirect
gcc read.c -o read
rishill10@ASUS16X:~/Practical9$ ./redirect
cat output.txt
This line is redirected to output.txt.
rishill10@ASUS16X:~/Practical9$ echo "Hello" > input.txt
./read < input.txt
You entered: Hello
rishill10@ASUS16X:~/Practical9$ echo "This is a sample file for testing file
copy operations in Linux." > input.txt
./read < input.txt
You entered: This
rishill10@ASUS16X:~/Practical9$ |
```